

บทที่ 2

ภาษา XML และภาษา Markup ที่เกี่ยวข้องกับการพัฒนาแผนที่

2.1 ภาษา XML (Extensible Markup Language)

XML ถูกกำหนดโดยกลุ่มทำงาน XML ของสถาบัน World Wide Web Consortium (W3C) กลุ่มทำงานกลุ่มนี้ได้บรรยายถึงภาษา XML ไว้ว่า Extensible Markup Language เป็นฟอร์แมตที่อธิบายถึงรายละเอียดของโครงสร้างและแบบของข้อมูล เป็นภาษาหรือชุดคำสั่งเกี่ยวกับข้อมูลบนเว็บที่ทำให้การพัฒนาในส่วนของการสร้างข้อมูลจากหลากหลายแอปพลิเคชัน มานำเสนอบนเครื่องเดสก์ท็อปด้วย XML จะทำให้การจัดการข้อมูลหรือเรียกใช้ข้อมูลจากแอปพลิเคชันต่างๆ จะเข้าสู่มาตรฐานเดียวกัน

XML และ HTML เป็นส่วนหนึ่งของภาษา SGML ซึ่ง XML จะให้รายละเอียดเกี่ยวกับข้อมูล เช่น ชื่อเมือง อุณหภูมิ ความกดอากาศ ส่วน HTML เป็นการกำหนด Tag ต่างๆ ที่จะกำหนดรูปแบบการแสดงผลของข้อมูลบนหน้าเว็บ ซึ่งข้อมูลจะสามารถแสดงออกมาได้หลายรูปแบบขึ้นอยู่กับข้อกำหนดของ HTML

2.1.1 ลักษณะโครงสร้างของ XML

XML เป็นการใช้อ็ความเพื่อบ่งบอกโครงสร้างของเอกสาร พิจารณาตัวอย่างรูปแบบโครงสร้างของหนังสือ เมื่อหนังสือประกอบด้วยจำนวนบท 2 บท ในแต่ละบทประกอบด้วยเนื้อความ

Begin Book

Begin Chapter 1

Text for Chapter 1

End Chapter 1

Begin Chapter 2

Text for Chapter 2

End Chapter 2

End Book

หนังสือที่มีอยู่ในปัจจุบันจะมีโครงสร้างที่มีรายละเอียดที่ซับซ้อนมากกว่านี้ เช่น บทนำ, สารบัญ เป็นต้น เช่นเดียวกัน ภายในส่วนเนื้อความยังประกอบด้วยโครงสร้างย่อยคือ ย่อหน้า (Paragraph) แต่ละย่อหน้ายังประกอบขึ้นจากประโยค คำและตัวอักษรด้วยลักษณะของเอกสาร XML นั้น สามารถอธิบายโดยใช้ตัวอย่างที่ 1 ได้ ดังนี้

ตัวอย่างที่ 1

```
<?xml version="1.0" encoding="windows-874"?>
<note>
  <to> Tiny </to>
  <from>Michael</from>
  <heading>กรุณาติดต่อ Mr.Michael Tiny</heading>
  <body>Mr.Tiny โทรศัพท์ทางไกลมาบอกว่ามีปัญหาเรื่องภาษีศุลกากร ติดต่อกลับ
</body>
</note>
```

บรรทัดที่ 1 นั้นหมายความว่าเราประกาศเอกสารนี้เป็นเอกสาร XML เวอร์ชันที่ 1.0 และมีการเข้ารหัสอักขระเป็น windows-874 เพื่อให้ใช้ภาษาไทยได้ บรรทัดที่ 2 คือ Element ตัวแรกหรือตัวแม่ของเอกสาร หรือที่เรียกว่า root element ที่ทุกๆ เอกสาร XML ต้องมีโดยที่เอกสารหนึ่งเอกสารมีได้เพียง root element เดียวเท่านั้น บรรทัดที่ 3 ถึง 6 มี element 4 ตัวคือ to, from, heading และ body ซึ่งเป็น element ลูกของ element note ส่วนบรรทัดสุดท้ายก็คือแท็กปิดของ element แม่ นั่นเองและทุกๆ element ในเอกสาร XML ต้องมีทั้งแท็กเปิดและแท็กปิด

2.1.1.1 Tag

tag ใน XML มีความหมายในลักษณะเดียวกับที่ใช้ใน HTML Tag คือข้อความที่อยู่ระหว่างสัญลักษณ์ < และ > ส่วนชนิดของ tag จะมีทั้ง tag เปิดและ tag ปิดแต่ก็จะมีที่เป็นแท็กเปิดและแท็กปิดอยู่ในตัวเดียวกันด้วยเช่น <ชื่อแท็ก /> ซึ่งแท็กเปิดเป็นตัวบอกว่าเริ่ม element และแท็กปิดเป็นตัวบอกว่าจบ element และความแตกต่างระหว่างแท็กเปิดกับแท็กปิดคือแท็กเปิดสามารถใส่ข้อมูลอธิบายเพิ่มเติมซึ่งเรียกว่า attribute แต่แท็กปิดไม่สามารถทำได้

2.1.1.2 Element

Element เป็นส่วนขยายอธิบายความหมายเพิ่มได้อีกและมีความสัมพันธ์แบบ element แม่กับ element ลูก ใน element สามารถใส่ข้อมูลเข้าไปได้โดยข้อมูลที่ใส่เข้าไประหว่างแท็กเปิด กับแท็กปิดเรียกว่า Content และข้อมูลที่ใส่เพิ่มเข้าไปในแท็กเปิดเพื่ออธิบายคุณสมบัติลักษณะของ element เพิ่มเรียกว่า Attribute ทุก element ในเอกสารต้องมีทั้งแท็กเปิดและแท็กปิด

2.1.1.4 Content

เนื้อหาหรือ Content ถือได้ว่าเป็นข้อมูลเพื่อใช้ในการแสดงให้ผู้อ่านเอกสารได้เห็น หรือ กล่าวอีกนัยหนึ่งคือ Content อยู่หลัง Tag เปิด และจบก่อนที่จะถึง Tag ปิดนั่นเอง

2.1.1.3 Attribute

Element ในเอกสาร XML สามารถมี Attribute ในแท็กเปิดได้และมีได้มากกว่า 1 ตัวเหมือนกับในเอกสาร HTML โดยที่ Attribute จะอธิบายถึงคุณสมบัติหรือบอกลักษณะของ element นั้นเพิ่ม คือถ้ากล่าวถึงสิ่งของหากเราไม่ได้ให้ความหมายเพิ่มเติมเราก็จะไม่มีทางรู้หรือแยกแยะสิ่งที่คล้ายๆ กันออกจากกันได้ เช่น ถ้าพูดถึง บท ในหนังสือ แค่นี้เป็นความหมายโดยรวม แต่ถ้าบอกว่าบทที่ 1 ในหนังสือ เลข 1 ในที่นี้คือความหมายเพิ่มเติมให้กับบท

2.1.2 ตัวกำหนดโครงสร้างของ XML ด้วย DTD (Document Type Definition)

Document Type Definition ใช้ในการกำหนดโครงสร้างของเอกสาร XML โดยเมื่อเปิดเอกสาร XML ด้วย DTD ตัวประมวลผลจะตรวจสอบเอกสารว่าถูกต้องตรงกับ DTD ที่ได้กำหนดโครงสร้างไว้หรือไม่ เช่น ถ้าไม่ประกาศ Element หรือ Attribute ไว้ใน DTD แล้วจะไม่สามารถใช้ Element หรือ Attribute นั้นๆ ในเอกสาร XML ได้

2.1.2.1 รูปแบบของ DTD

DTD มีรูปแบบทั่วไปตามนี้ `<!DOCTYPE Name DTD> Name` ตามตัวอย่างเป็นการกำหนดชื่อของ Document Element (หรือ Root Element) ชื่อของ Document Element จะต้องตรงกับชื่อที่กำหนดไว้ที่นั่น DTD สามารถบรรจุชนิดของการประกาศ Markup ไว้ดังนี้

การประกาศ Element Type

โดยมีรูปแบบการประกาศ Element Type ดังนี้

```
<! ELEMENT ชื่อelement (รายละเอียดในelement)>
```

รายละเอียดใน Element เป็นที่กำหนดว่าภายใน Element สามารถบรรจุอะไรได้บ้าง ด้านล่างนี้จะเป็นตัวอย่างของเอกสาร XML ที่มี Element Type 2 ชนิด และ Element Type ที่ประกาศชื่อ COLLECTION แสดงให้เห็นว่าสามารถบรรจุ Element ที่ชื่อ CD ได้ มากกว่า 1 Element และ Element Type ที่ชื่อ CD กำหนดว่าสามารถบรรจุได้เฉพาะข้อมูลประเภท Character Data เท่านั้น

```
<?xml version="1.0"?>
```

```
<!DOCTYPE COLLECTION
```

```
[ <!ELEMENT COLLECTION (CD) +>
```

```
<!ELEMENT CD (#PCDATA)>
```

```
]
```

```
>
```

```
<COLLECTION>
```

```
<CD>Mozart Violin Concertos 1, 2, and 3</CD>
```

```
<CD>Telemann Trumpet Concertos</CD>
```

```
<CD>Handel Concerti Grossi</CD>
```

```
</COLLECTION>
```

การประกาศ Attribute

โดยมีรูปแบบของการประกาศดังนี้

<! ATTLIST ชื่อของ element ชื่อของแอตทริบิวต์ ชนิดของแอตทริบิวต์ ค่าดีฟอลต์>

ชนิดของแอตทริบิวต์มีได้หลายอย่าง ดังตารางที่ 2-1

ชนิดของแอตทริบิวต์	คำอธิบาย
CDATA	ค่าที่เป็นข้อมูลแบบ Character Data (ไม่กระจายค่าในพาสเซอร์)
(eval eval ...)	ค่านี้จะป็นกลุ่มของค่าที่เตรียมไว้ให้เลือก
ID	ค่านี้เป็นเลขประจำตัวที่ไม่ซ้ำกับใคร
IDREF	ค่านี้เป็นเลขประจำตัวของelementหนึ่ง
IDREFS	ค่านี้เป็นลิสต์ของเลขประจำตัวอื่นๆ
NMTOKEN	ค่านี้เป็นชื่อที่ถูกต้องตามหลัก XML
NMTOKENS	ค่านี้เป็นลิสต์ของชื่อที่ถูกต้องตามหลัก XML
ENTITY	ค่านี้เป็นเอนติตี้
ENTITIES	ค่านี้เป็นลิสต์ของเอนติตี้
NOTATION	ค่านี้เป็นชื่อของ Notation ที่ได้ประกาศไว้
Xml:	ค่านี้เป็นค่าที่มีการกำหนดไว้ก่อนแล้ว

ตารางที่ 2-1 แสดงชนิดของแอตทริบิวต์ที่สามารถใช้ในการประกาศแอตทริบิวต์

ส่วนที่เป็นค่าดีฟอลต์สามารถกำหนดได้ดังตารางที่ 2-2

ค่าที่กำหนด	คำอธิบาย
#DEFAULT value	แอตทริบิวต์นี้มีค่าดีฟอลต์และระบุมาให้
#REQUIRED	ต้องมีค่าแอตทริบิวต์ให้กับelement
#IMPLIED	ค่าแอตทริบิวต์อาจไม่จำเป็นต้องให้ไว้ก็ได้
#FIXED value	ค่าของแอตทริบิวต์เป็นค่าที่กำหนดไว้ตายตัว ผู้ใช้เปลี่ยนไม่ได้

ตารางที่ 2-2 แสดงค่าที่สามารถได้ในส่วนที่เป็นค่าดีฟอลต์ในการประกาศแอตทริบิวต์

2.1.3 การแปลงรูปแบบเอกสาร XML (Transformation XML)

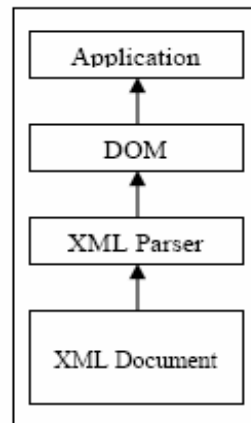
XSL ซึ่งย่อมาจาก Extensible Style sheet Language (XSL) เป็นภาษาที่ใช้ในการแปลงจากรูปแบบ(Transformation) ของเอกสาร XML ไปอยู่ในรูปแบบอื่นๆ และในขณะเดียวกันก็ทำการประยุกต์รูปแบบการจัดข้อความ (style) ด้วย XSL stylesheet (สไตล์ชีต) ถูกเขียนในรูปแบบที่เหมาะสมกับ XML แต่ต้องทำการกำหนดลำดับของ Element ไว้ล่วงหน้าในการกำหนดสิ่งที่ต้องการให้กระทำในการแปลงรูปแบบโดยความหมายจะถูกกำหนดโดยตัวประมวลผล XSL เอง ซึ่งควรจะเป็นไปตามมาตรฐานของ W3C ดังนั้นจาก XSL ซึ่งเป็นแอปพลิเคชันหนึ่งของ XML และอยู่ในรูปแบบที่เหมาะสมกับ XML ดังนั้นจึงสามารถประมวลผล XSL สไตล์ชีตได้เหมือนกับเอกสาร XML ครั้งแรก XSL เป็นข้อเสนอที่ถูกรับรู้ทั้งทางที่การแปลงรูปแบบ XML และไวยากรณ์ในการจัดการรูปแบบบนแพลตฟอร์มที่เป็นอิสระกัน (platform-independent styling grammar) แต่อย่างไรก็ตามในส่วนของการแปลงรูปแบบก้าวหน้าอย่างรวดเร็วและเป็นประโยชน์อย่างมาก ขณะที่ไม่มีแอปพลิเคชันในส่วนของการจัดรูปแบบมากนักและส่วนใหญ่จะกล่าวว่ามีประโยชน์ทางด้านการทำงานน้อย ดังนั้นจากข้อเสนอในตอนแรกจึงสามารถแบ่งออกได้เป็น 2 ส่วนที่แตกต่างกันคือ ส่วนในการแปลงรูปแบบเอกสาร ซึ่งปัจจุบันถูกเรียกว่า XSLT (XSL Transformation Language) แต่ในที่นี้จะพิจารณาเฉพาะในส่วนของการแปลงรูปแบบเท่านั้นเพื่อแปลงรูปแบบจากเอกสาร XML แบบหนึ่งไปเป็น XML อีกแบบหนึ่ง เช่นใช้ในการแปลงเอกสาร GML ไปเป็น SVG

2.1.3.1 The XSL Transformation Language (XSLT)

XSLT ถูกกำหนดโดยองค์กร W3C ในการกำหนดเซต Element ของ XML ที่สามารถจะถูกใช้ในการสร้างสไตล์ชีตที่แปลงเอกสาร XML ให้อยู่ในรูปแบบใดๆ ซึ่งในปัจจุบันนี้ใช้ในการแปลง XML รูปแบบหนึ่งไปเป็นอีกรูปแบบหนึ่ง โดยการทำงานจะทำโดยนำเอกสาร XML ที่เหมาะสมกับ XML นั้นไปทำการแปลงรูปแบบเพื่อสร้างเอกสารเอาต์พุต ซึ่งขึ้นอยู่กับคำสั่งและเนื้อหาภายใน สไตล์ชีตจึงสามารถสร้างรูปแบบเอาต์พุตใดๆ ก็ได้ตามที่ต้องการในการทำงาน

2.1.4 DOM (Document Object Model)

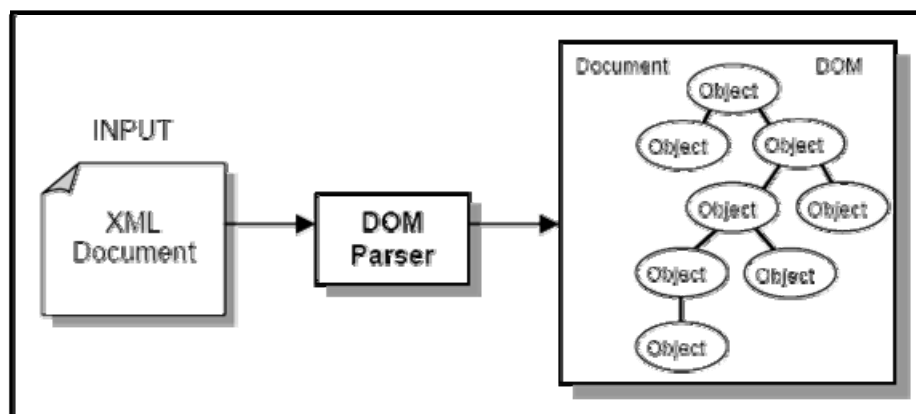
การใช้งาน XML นั้นเราสามารถที่จะทำอะไรกับข้อมูลในแอปพลิเคชันที่เราเขียนขึ้นได้ ไม่ใช่เพียงแค่การเข้าถึงเท่านั้นแต่ยังรวมถึงการแก้ไขและเพิ่มเติมเอกสาร XML ที่เรามีอยู่อีกด้วย จึงเกิดมี Document Object Model (DOM) ขึ้นเพื่อช่วยในการทำงานกับเอกสาร XML ทั้งหมด DOM มักจะถูกใส่เข้าไปเป็นชั้นที่กั้นกลางระหว่าง XML parser กับแอปพลิเคชันที่ต้องการใช้ข้อมูลในเอกสาร XML ซึ่งหมายความว่า parser จะอ่านข้อมูลจากเอกสาร XML ให้กับ DOM จากนั้น DOM จะถูกใช้โดยแอปพลิเคชันระดับสูงกว่า แอปพลิเคชันสามารถทำงานกับเอกสาร XML ได้ตามที่ต้องการ รวมถึงการใส่ DOM เข้าไปใน DOM ตัวอื่น ซึ่งในแต่ละภาษาก็จะมีแบบจำลอง DOM เป็นของตนเอง



รูปที่ 2-1 การใช้งาน DOM

DOM Parsing

DOM (Document Object Model) เป็นการทำงานแบบ Tree-Based parser และใช้การเข้าถึงข้อมูลด้วยวิธีการ Random-Access คือจะประมวลโครงสร้างของเอกสาร XML ให้เป็นโครงสร้างต้นไม้(Tree structure) แบบ hierarchical object model ซึ่งในโครงสร้างต้นไม้จะประกอบไปด้วยอีลิเมนต์โหนด (element node) โดยภายในแต่ละอีลิเมนต์โหนด ประกอบไปด้วยข้อมูลที่เกี่ยวข้องกับอีลิเมนต์นั้นตามโครงสร้างและข้อมูลในเอกสาร XML ซึ่งจะประกอบไปด้วยข้อมูลทั้งหมดของเอกสาร XML เพื่อให้แอปพลิเคชันสามารถเข้าหาจุดต่างๆ ของ tree structure ได้ โดยที่ DOM จะโหลด XML ทั้งหมด (ข้อมูลใน Root Node ทั้งหมด) เข้ามาเป็น Tree ในหน่วยความจำหลักของคอมพิวเตอร์ (memory) ก่อนจึงทำงานได้ดังแสดงในรูปที่ 2-2



รูปที่ 2-2 แสดงวิธีการเข้าถึงข้อมูลโดยวิธี DOM Parsing

2.2 ภาษา Markup Language ที่เกี่ยวข้องกับการพัฒนาแผนที่

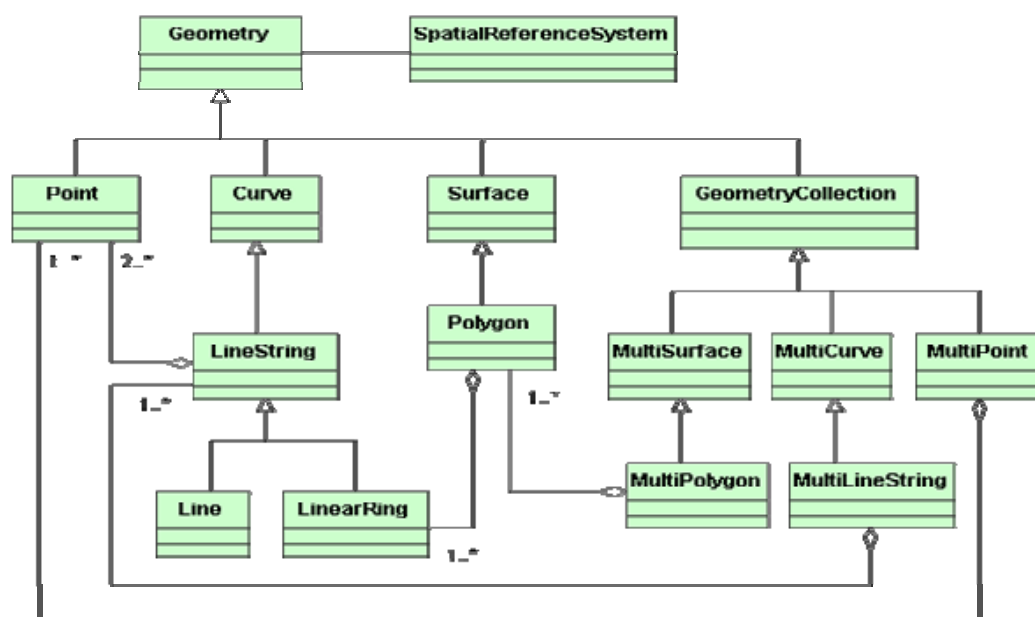
องค์กร OGC (OpenGIS Consortium) ได้กำหนดรูปแบบในการจัดเก็บและการแลกเปลี่ยนข้อมูลทางด้านภูมิศาสตร์ พร้อมทั้งความสัมพันธ์และลักษณะประจำตัวของข้อมูล ในรูปแบบที่เรียกว่า GML (Geography Markup Language) ซึ่งเป็นภาษา Markup Language รูปแบบหนึ่งที่มีพื้นฐานโครงสร้างที่อยู่ในรูปแบบของ XML (Extensible Markup Language) แต่รูปแบบการจัดเก็บและแลกเปลี่ยนข้อมูลทางด้านภูมิศาสตร์ในรูปแบบของ GML นั้นก็เป็นเพียงแค่ตัวข้อมูลที่ใช้ในการจัดเก็บเท่านั้น ไม่สามารถที่จะนำมาแสดงผลได้ ซึ่งการจะนำมาแสดงผลบน web browser ได้นั้นจำเป็นต้องแปลงรูปแบบให้ไปอยู่ในรูปแบบที่สามารถแสดงผลได้ก่อน ซึ่งหนึ่งวิธีในการแสดงผลของ GML บน web browser นั้นคือการแปลงให้อยู่ในรูปแบบของ SVG (Scalable Vector Graphics Language) ซึ่งเป็นภาษา Markup Language อีกรูปแบบหนึ่งที่ใช้ในการแสดงผลได้

2.2.1 ภาษา GML (Geography Markup Language)

ภาษา GML เป็นรูปแบบหนึ่งของเอกสาร XML ที่ถูกออกแบบขึ้นเพื่อการจัดเก็บและการแลกเปลี่ยนข้อมูลทางภูมิศาสตร์ ทั้งทางด้านความสัมพันธ์และลักษณะประจำตัวของข้อมูล ในรูปแบบของ feature ที่แตกต่างกันและสามารถกำหนดวิธีที่จะแสดงข้อมูลทางภูมิศาสตร์นั้น โดยใช้ข้อกำหนดของ GML ที่ถูกรับรองโดยองค์กร OGC (OpenGIS Consortium) ในการอธิบายข้อมูลเชิงพื้นที่เพื่อให้ผู้ที่ทำงานด้าน GIS (Geographic Information System) ใช้เป็นพื้นฐานในการออกแบบพัฒนาแผนที่ในรูปแบบต่างๆ

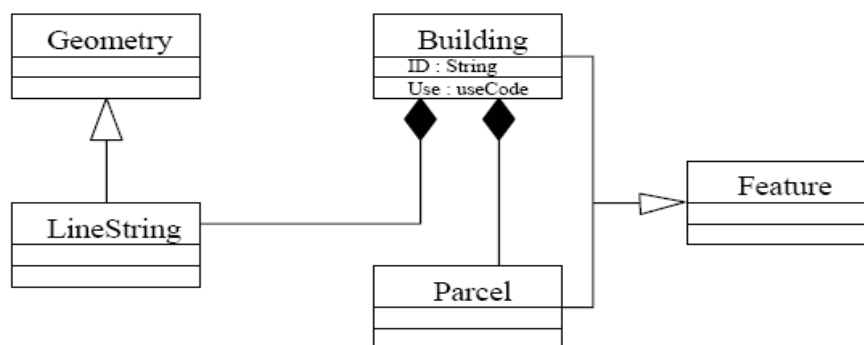
2.2.1.1 GML Basic concepts

GML เป็นรูปแบบหนึ่งของเอกสาร XML ที่ถูกออกแบบขึ้นเพื่อการจัดเก็บและการแลกเปลี่ยนข้อมูลทางภูมิศาสตร์



รูปที่ 2-3 The geometry model for simple feature (Open GIS Consortium, 2001)

องค์ประกอบทางเรขาคณิต (Geographic feature) จะถูกอธิบายด้วยคุณสมบัติต่างๆ ซึ่งคุณสมบัติเหล่านี้อาจจะประกอบด้วยคุณสมบัติที่เกี่ยวข้องกับระยะทาง โดยอธิบายตำแหน่งและรูปร่างของ feature นั้นๆ แต่ละ feature จะมีชนิดของ feature ซึ่งเทียบเท่ากับคลาสใน object โมเดล ดังนั้น Class definition จึงเป็นส่วนที่กำหนดคุณสมบัติของแต่ละ feature จำเป็นต้องมี เช่น อาคารที่ต้องมีการกำหนดให้มีชื่อของอาคาร ซึ่งเป็นข้อมูลประเภท String และเส้นรอบนอกซึ่งกำหนดให้เป็นรูปวงแหวน (LinearRing) ซึ่งเป็นคุณสมบัติทางภูมิศาสตร์ โดยรูปด้านบนอธิบายถึง Geographic feature โดยใช้ UML



รูปที่ 2-4 Features and properties

2.2.1.2 Basic geometric properties

รูปทรงเรขาคณิตพื้นฐานที่มีการนิยามไว้ใน GML มีดังตารางที่ 2-3

ชื่อ	อธิบาย	ชนิดของเรขาคณิต
boundedBy	-	Box
pointProperty	location, position, centerOf	Point
lineStringProperty	CenterLineOf	LineString
polygonProperty	coverage	Polygon
geometryProperty	-	-
multiPointProperty	multiLocation, multiPosition, multiCenterOf	MultiPoint
multiLineStringProperty	multiCenterLineOf	MultiLineString
multiPolygonProperty	multiCoverage	MultiPolygon
multiGeometryProperty	-	MultiGeometry

ตารางที่ 2-3 แสดงรูปทรงเรขาคณิตพื้นฐานที่ถูกนิยามไว้ใน GML

2.2.1.3 Encoding Geometry

สิ่งที่สำคัญที่สุดในมาตรฐานของการแปลงข้อมูลทางภูมิศาสตร์ คือความสามารถในการแสดงรูปร่างทางเรขาคณิต ในการแสดงภาพทางภูมิศาสตร์ที่เกี่ยวข้องกับระยะทางโดยใช้ GML 2.0 นั้นจำเป็นที่จะต้องอ้างอิงถึง XML Schema ทั้งหมด 2 Schema คือ GML Feature Schema และ GML Geometry Schema

1. GML Feature Collections

1. ภาษา GML ใช้ feature อธิบายองค์ประกอบต่างๆ ของภูมิประเทศในรูปแบบของ Geographic entities
2. feature ประกอบด้วย list ของคุณสมบัติต่างๆ ไป ซึ่งประกอบด้วยชื่อของข้อมูล ชนิดของข้อมูล ค่าของข้อมูล และคุณสมบัติต่างๆทางภูมิศาสตร์เช่น lines, points, curves, polygons
3. feature สามารถสร้างขึ้นจาก feature อื่นๆ หรือกำหนดชนิดของข้อมูลเป็น feature อื่นๆ ได้

ตามหลักของ OGC Simple Feature Model, GML จะรองรับการแสดงผลขององค์ประกอบทางเรขาคณิต คือ จุด (point), เส้น (LineString), วงแหวน (LinearRing), รูปหลายเหลี่ยม (Polygons)

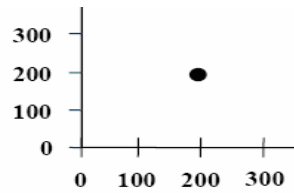
และโดยรูปแบบพื้นฐานที่มีมาให้ เราสามารถเพิ่มเติม MultiPoint, MultiString, MultiPolygon และ MultiGeometry นอกจากองค์ประกอบทางเรขาคณิตเหล่านี้แล้ว GML ยังประกอบด้วย <coordinates> หรือ <coord> ซึ่งเป็น tag ที่ใช้ในการอ้างอิงถึงจุดคู่ลำดับหรือตำแหน่ง และ <Box> ใช้ในการนิยามองค์ประกอบที่มีพื้นที่เป็นรูปสี่เหลี่ยม

ส่วนที่สำคัญของ GIS คือวิธีการในการอ้างอิงถึงองค์ประกอบทางภูมิศาสตร์ต่างๆ บนพื้นโลกคู่ลำดับ (coordinate) สำหรับรูปเรขาคณิตที่เราใช้แสดงแทนบนแผนที่นั้นจะอ้างอิงโดย Spatial Reference System (SRS) และทุกองค์ประกอบประกอบจะต้องระบุ SRS หรืออีกวิธีหนึ่งคือระบุ GID Attribute ซึ่งเป็น Attribute ที่จะแสดงค่าเฉพาะตัวทางภูมิศาสตร์ของแต่ละองค์ประกอบเรขาคณิตนั้น

Points เป็นรูปเรขาคณิตพื้นฐานที่ ประกอบด้วยคู่ลำดับเพียงคู่ลำดับเดียว multipoint เป็นเซตที่ประกอบด้วยจุดหลายๆ จุด

ตัวอย่าง Code

```
<Point GID="" srsName="gml_srs">
  <coord><x>200.0</x><y>200.0</y></coord>
</Point>
```

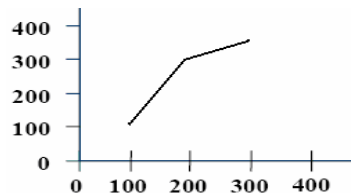


รูปที่ 2-5 แสดงรูปที่สร้างโดย Point

LineString ประกอบด้วย list ของคู่ลำดับแสดงที่ส่วนของเส้นตรงหลายเส้นเชื่อมต่อกัน
รูปด้านล่าง LineString จะต้องประกอบด้วยคู่ลำดับอย่างน้อย 2 คู่

ตัวอย่าง Code

```
<LineString gid="2468" srsName="gml_srs">
  <coord><x>100.0</x><y>100.0</y></coord>
  <coord><x>200.0</x><y>300.0</y></coord>
  <coord><x>300.0</x><y>350.0</y></coord>
</LineString>
```



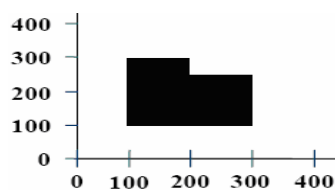
รูปที่ 2-6 แสดงรูปที่สร้างโดย LineString

LinearRing เป็นรูปปิด ประกอบด้วย list ของคู่ลำดับที่แสดงถึงส่วนของเส้นตรง คู่ลำดับสุดท้ายจะต้องเท่ากับคู่ลำดับแรก และต้องประกอบด้วยคู่ลำดับอย่างน้อย 4 คู่

Polygons เป็นพื้นที่ผิวของรูปเป็นเซตของ LinearRing โดยขอบนอกของ Polygon คือ LinearRing วงแรกและ LinearRing วงที่เหลือจะซ้อนเรียงกันภายในของวงแรก

ตัวอย่าง Code

```
<Polygon gid="1234" srsName="gml_srs">
  <outerBoundaryIs>
    <LinearRing>
      <coord><x>100.0</x><y>100.0</y>
      <coord><x>100.0</x><y>300.0</y>
      <coord><x>200.0</x><y>300.0</y>
      <coord><x>200.0</x><y>200.0</y>
      <coord><x>300.0</x><y>200.0</y>
      <coord><x>300.0</x><y>100.0</y>
      <coord><x>100.0</x><y>100.0</y>
    </LinearRing>
  </outerBoundaryIs>
</Polygon>
```



รูปที่ 2-7 แสดงรูปที่สร้างโดย Polygon

2. GML application schema

Schema คือรูปแบบการจัดโครงสร้างและความสัมพันธ์ของข้อมูล โดยภาษา GML นั้นกำหนดให้ผู้ใช้ต้องกำหนด GML Application schema ซึ่งเขียนโดยใช้หลักการของ XML Schema และเป็นไปตามข้อบังคับของ GML Application Schema ประกอบด้วย

- Annotations ซึ่งเป็นส่วนประกาศหรือส่วนที่ใช้การอธิบาย schema นั้น
- Simple Type คือชนิดของข้อมูลที่ใช้รูปแบบของข้อมูลที่ GML กำหนดมาให้แล้ว
- Complex Type คือชนิดของข้อมูลที่สร้างขึ้นใหม่โดยอาจจะเกิดจากการใช้ simple type หลายๆ ตัวมาประกอบกัน หรือเกิดจาก Complex type ชนิดอื่นก็ได้
- Element
- Attribute

ตัวอย่าง GML Feature Type Schema

```
<element name="RANET Radio Station" type="RadioSchema"/>
<complexType name="RadioSchema">
  <sequence>
    <element name="villageName" type="string"/>
    <element name="countryName" type="string"/>
    <element name="numWorldSpace" type="integer"/>
    <element name="numFreeplay" type="integer"/>
  </sequence>
</complexType>
```

ข้อดีของการใช้ GML ในการอธิบายแผนที่

1. ได้แผนที่ที่มีคุณภาพดีกว่า เนื่องจาก GML จะเปลี่ยนข้อมูลทางภูมิศาสตร์ทั้งหมดให้อยู่ใน รูปของ Feature หรือ object ซึ่งสามารถนำไปแสดงผลในรูปแบบที่ไม่จำกัดความละเอียดในการแสดงผลได้
2. สามารถใช้งานได้บนบราวเซอร์โดยไม่ต้องซื้อโปรแกรมที่ใช้แสดงผลทางฝั่งไคลเอนท์ เมื่อข้อมูล GML ถูกรับโดยฝั่งไคลเอนท์จะถูกเปลี่ยนให้อยู่ในรูปของภาพแผนที่บนบราวเซอร์โดยใช้ภาษา SVG (Scalable Vector Graphics) ในการแสดงออกมาเป็นภาพ ซึ่งเราสามารถแสดงแผนที่บนบราวเซอร์โดยไม่จำเป็นต้องติดตั้งซอฟต์แวร์อื่นๆ เพิ่มเติมหากบราวเซอร์สามารถรองรับภาพแบบเวกเตอร์ (Vector graphics) ได้ และเราสามารถดาวน์โหลด SVG Viewer ซึ่งเป็นปลั๊กอินที่ใช้สำหรับดูภาพแบบ SVG ของบริษัท Adobe ได้โดยไม่ต้องเสียค่าใช้จ่ายใดๆ

3. สามารถสร้างแผนที่ที่มีรูปแบบเฉพาะได้ เนื่องจาก GML จะเก็บแค่ข้อมูลของแผนที่ เช่นที่ตั้งของสถานที่นั้น, ลักษณะทางภูมิศาสตร์ ฉะนั้นแล้วการแสดงผลก็ขึ้นอยู่กับความพอใจของ ผู้พัฒนาระบบนั้นๆ
4. สามารถทำการแก้ไขแผนที่ได้ แผนที่ที่เขียนโดย GML จะถูกดาวน์โหลดและทำการ render บนบราวเซอร์ซึ่งเมื่อ GML ถูกเปลี่ยนเป็น SVG ผู้ใช้จะสามารถใช้โปรแกรมบนไคลเอนท์ในการเพิ่มเติมตัวอักษรหรือทำการเปลี่ยนแปลงภาพบนแผนที่ และสามารถบันทึกเก็บเป็นไฟล์ภาพเพื่อนำไปใช้ในกรณีอื่นได้
5. สามารถกำหนดเนื้อหาที่ต้องการแสดงหรือต้องการได้รับ เพราะ GML เป็นภาษาที่อ้างอิงถึง feature ทำให้ผู้ใช้สามารถกำหนดได้ว่าต้องการให้มี feature ใดแสดงบนแผนที่บ้างซึ่งจะช่วยลดระยะเวลาในการรับ
6. สามารถแสดงผลได้บนทุกอุปกรณ์ที่ใช้บราวเซอร์ได้ ข้อมูลทางภูมิศาสตร์ใน GML นั้นสามารถส่งไปยังอุปกรณ์ชนิดใดก็ได้ที่สามารถรองรับ XML การแสดงผลแบบ XML ได้ เช่น เครื่องพีดีเอหรือโทรศัพท์มือถือ ทำให้แผนที่นั้นสามารถใช้งานได้อย่างกว้างขวางและหลากหลายยิ่งขึ้น
7. สามารถเชื่อมโยง link ลงไปยังในแผนที่ได้ ประโยชน์ที่สำคัญของ GML คือผู้ใช้สามารถฝัง URL ลงใน feature ที่มีในแผนที่ได้ ความสามารถนี้ทำให้เราสามารถออกแบบแผนที่ให้มีความแปลกใหม่และทันสมัยยิ่งขึ้น
8. ทำการ query ได้ดียิ่งขึ้น หากผู้ใช้ต้องการที่จะคลิกไปบน feature บนแผนที่และค้นหาข้อมูลเพิ่มเติมเกี่ยวกับ feature นั้นๆ (เช่น ชื่อหรือความลึกของแม่น้ำ) ซึ่งไม่สามารถทำได้บนแผนที่ที่สร้างขึ้นโดยใช้ไฟล์ภาพ (gif/jpg) สำหรับแผนที่ที่ใช้ GML นั้น เราจะสามารถระบุ feature ที่ถูกคลิกนั้นได้

2.2.2 ภาษา SVG (Scalable Vector Graphics Language)

SVG เป็นภาษาที่ใช้อธิบายภาพกราฟิก 2 มิติและอยู่ในรูปแบบ XML และกราฟิกที่อธิบายนั้นเป็นแบบเวกเตอร์ด้วย สามารถจัดกลุ่ม กำหนดลักษณะ และอื่นๆ ของการอธิบายรูปกราฟิก และสามารถที่จะทำให้กราฟิกนั้นมีการทำงานที่มีการปฏิสัมพันธ์กับผู้ใช้งานโดยใช้ภาษาจาวาสคริปต์ต่างๆเข้าช่วยและสามารถทำงานในรูปแบบแอนิเมชันได้อีกด้วย

และเนื่องจาก SVG เป็น XML ดังนั้นจึงมีการสร้าง DOM สำหรับ SVG ขึ้นมาโดยอ้างอิงตามมาตรฐานของ W3C ซึ่งทำให้ SVG มีคุณสมบัติไดนามิกและการปฏิสัมพันธ์ตามไปด้วย แม้แต่การจัดการเหตุการณ์ที่เกิดขึ้นที่ตัวกราฟิก อย่างเช่น onmouseover() หรือ onclick() ก็สามารถกำหนดลงในวัตถุของ SVG ได้อีกด้วย โดยรายละเอียดเพิ่มเติมสามารถศึกษาได้จาก

<http://www.w3.org/TR/SVG/index.html#minitoc>

2.2.2.1 การอธิบายภาพโดยใช้ภาษา SVG

ลักษณะการเขียนภาษา SVG มีลักษณะเป็น XML ซึ่งต้องมี tag เปิดและปิด และมี tag ที่อยู่ใน tag ด้วย ซึ่งจะสังเกตได้ว่า SVG ไฟล์ จะต้องมีการมี <svg> อยู่เป็น tag นอกสุดหรือเป็น root element ซึ่งมีตัวอย่างดังต่อไปนี้

```
<?xml version="1.0" encoding="iso-8859-1"?>
<!DOCTYPE svg PUBLIC "-//W3C//DTD SVG 20000303 Stylable//EN"
"http://www.w3.org/TR/2003/WD-SVG-20000303/DTD/svg-20000303-
stylable.dtd">
<svg xml:space="preserve" width="5.5in" height=".5in">
...
...
...
</svg>
```

ในบรรทัดแรกจะเป็นการบอกให้ผู้ทึ่ นำ SVG ไฟล์นี้ไปอ่านทราบว่าเอกสารต่อไปนี้อ้างอิงจากมาตรฐาน XML เวอร์ชัน 1.0 โดยข้อมูลที่อยู่ในแต่ละแท็กจะใช้มาตรฐาน iso-8859-1 ในการเขียนบรรทัดที่สองจะอ้างอิงถึง DTD หรือ Document Type Definition (DTD เป็นเอกสารที่ใช้บอกถึงชนิดของข้อมูลที่จะใช้ใน XML และยังระบุอีกด้วยว่าเอกสารนั้นๆ มีโครงสร้างเป็นอย่างไร) ใน <svg> Element สามารถที่จะบรรจุ text, shapes และ paths ได้ เช่น

Vector Graphic Shapes

<rect> : ใช้สำหรับสร้างและปรับแต่งรูปสี่เหลี่ยม

ตัวอย่าง

```
< rect x="20" y="20" rx="20" ry="20" width="250"
height="100" style="fill:red; stroke:black; stroke-width:5; opacity:0.5"/>
```

คำอธิบาย แอดทริบิวต์

x, y : ใช้กำหนดตำแหน่งของรูปสี่เหลี่ยมที่สร้าง

rx, ry : ใช้กำหนดส่วนโค้งบริเวณมุมสี่เหลี่ยม

width, height : ใช้กำหนดขนาดความกว้างและความสูงตามลำดับ

style : ใช้ในการกำหนดลักษณะของวัตถุ

fill, stroke : ใช้กำหนดสีของวัตถุและสีของเส้นขอบวัตถุตามลำดับ

stroke-width : ใช้กำหนดความหนาของเส้น

opacity : ใช้กำหนดความโปร่งใสของวัตถุมีค่าระหว่าง 0-1

<circle> : ใช้สำหรับสร้างและปรับแต่งรูปวงกลม

ตัวอย่าง

```
<circle cx="100" cy="50" r="40" stroke="black"stroke-width="2" fill="red />
```

คำอธิบายแอตทริบิวต์

cx, cy : ใช้กำหนดจุดศูนย์กลางของวงกลม หากไม่กำหนดจะมีค่าเป็น 0, 0

r : ใช้กำหนดรัศมีของวงกลม

<ellipse> : ใช้สำหรับสร้างและปรับแต่งรูปวงรี

ตัวอย่าง

```
<ellipse cx="300" cy="150" rx="200" ry="80" style="fill:rgb(200,100,50);
stroke:rgb(0,0,100);stroke-width:2" />
```

คำอธิบาย แอตทริบิวต์

cx, cy : ใช้กำหนดจุดศูนย์กลางของวงกลม หากไม่กำหนดจะมีค่าเป็น 0, 0

rx, ry : ใช้กำหนดรัศมีในแนวนอนและแนวตั้งตามลำดับ

<line> : ใช้สำหรับสร้างและปรับแต่งเส้นตรง

ตัวอย่าง

```
<line x1="0" y1="0" x2="300" y2="300" style="stroke:rgb(99,99,99);stroke-width:2" />
```

คำอธิบาย แอตทริบิวต์

x1, y1 : ใช้กำหนดตำแหน่งจุดเริ่มต้นของเส้นตรง

x2, y2 : ใช้กำหนดตำแหน่งจุดสิ้นสุดของเส้นตรง

<polygon> : ใช้สำหรับสร้างและปรับแต่งรูปหลายเหลี่ยม

ตัวอย่าง

```
<polygon points="220,100 300,210 170,250" style="fill:#cccccc;stroke:#000000;
stroke-width:1" />
```

คำอธิบาย แอตทริบิวต์

point ใช้กำหนดตำแหน่งของมุมแต่ละมุม (x,y)

<polyline> : ใช้สำหรับสร้างและปรับแต่งรูปเส้นตรงหลายเส้นเชื่อมต่อกัน

ตัวอย่าง

```
<polyline point="0,0 0,20 20,20 20,40 40,40 40,60" style="fill:#cccccc;stroke:#000000;stroke-width:1" />
```

คำอธิบาย แอตทริบิวต์

point ใช้กำหนดตำแหน่งจุดเชื่อมต่อของเส้น (x,y)

<path> : ใช้สำหรับสร้างและปรับแต่งรูปร่างต่างๆโดยจะมีคำสั่งที่ใช้ในการสร้างรูปร่างดังนี้

M = moveto	S=smooth curveto
- A = elliptical Arc	L = lineto
- V = vertical lineto	T = smooth quadratic Belzier curveto
Q=quadratic Belzier curve	C = curveto
- H = horizontal lineto	Z = closepath

ตารางที่ 2-4 แสดงคำสั่งที่ใช้ในการสร้างรูปร่าง

ตัวอย่าง

```
<path d="M250 150 L150 350 L350 350 Z" />
```

คำอธิบาย

จากตัวอย่างเป็นการสร้าง path โดยเริ่มต้นที่จุด 250,150 จากนั้นลากเส้นตรงไปยังจุด 150,350 จากนั้นลากเส้นตรงไปยังจุด 350,350 จากนั้นทำการปิด path โดยกลับไปจุด 250,150

2.2.2.2 Element ที่ใช้อธิบายรูปร่างพื้นฐาน

Element ที่ใช้อธิบายรูปร่างพื้นฐานมีดังนี้

- rect Element ใช้อธิบายรูปสี่เหลี่ยมซึ่งในแท็กเปิดของ element นี้ก็จะมีแอตทริบิวต์ที่จะคอยอธิบายถึงลักษณะของรูปสี่เหลี่ยมเพิ่มเติมว่ามีลักษณะอย่างไรบ้างเช่น
- circle Element ใช้อธิบายรูปร่างกลมว่าจะมีลักษณะ ขนาดของรัศมี สี และอื่นๆ
- ellipse Element บอกถึงลักษณะของรูปร่างรี
- line Element ใช้อธิบายถึงลักษณะของเส้นขนาดเส้นใหญ่ขนาดไหนมีสีอะไร
- polygon Element ใช้อธิบายถึงลักษณะของรูปหลายเหลี่ยม

2.2.2.3 ประโยชน์ในการใช้ SVG

- SVG สามารถอ่านและแก้ไขได้โดยโปรแกรมหลายโปรแกรม เช่น notepad
- สามารถเปลี่ยนขนาดของภาพได้โดยคุณภาพของภาพไม่ลดลง
- SVG สามารถทำงานร่วมกับภาษาจาวาได้
- SVG เป็นมาตรฐานที่เปิดเผยให้คนทั่วไปทราบ
- ข้อความใน SVG มีความสามารถในการเลือกและค้นหาจึงเหมาะแก่การนำมาทำเป็นแผนที่

โดย element และ attribute ของ SVG ที่ใช้งานบ่อยมีดังต่อไปนี้

SVG Element

SVG	Action
<svg>	เป็น tag ที่ใช้ในการกำหนดพื้นที่ทั้งหมดของภาพและเป็น tag ที่เก็บ tag ต่างๆ ของ SVG
<path>	ใช้ในการวาดเส้น
<rect>	ใช้ในการวาดสี่เหลี่ยม
<g>	ใช้ในการรวม tag ต่างๆ ให้อยู่ในกลุ่มเดียวกัน
<desc>	ใช้ในการเพิ่มเติมข้อมูลหรือคำอธิบาย
<defs>	ใช้ในการกำหนดค่าต่างๆ
<script>	ใช้ในการเขียน script จะเขียนใน tag นี้
<text>	ใช้ในการแสดงผลแบบอักษร
<cursor>	ใช้ในการเปลี่ยนรูปของ mouse cursor
<line>	ใช้ในการวาดเส้นอย่างง่าย
<textPath>	ใช้ในการเขียนอักษรให้เป็นไปตามเส้นทางที่กำหนด
<image>	ใช้ในการแสดงรูปภาพในรูปแบบต่างๆ เช่น jpeg, gif, png เป็นต้น

ตารางที่ 2 -5 SVG Element

SVG Attribute

Attribute	Action
fill	จัดการกับสีของ object นั้นๆ
id	ใช้ในการแยกแยะ object ต่างๆ
Width	ขนาดความกว้างของ object
Height	ขนาดความสูงของ object
X	ตำแหน่งพิกัด x
Y	ตำแหน่งพิกัด y
Xmlns	ใช้ในการอ้างอิงถึง namespace
Visibility	กำหนดการปรากฏอยู่ของ object
Style	กำหนดลักษณะของ object
point-event	กำหนดการปฏิสัมพันธ์กับ object
viewBox	กำหนดขนาดในการแสดงผลของภาพ
stroke-width	กำหนดขนาดของเส้น
Stroke	กำหนดสีของเส้น
zoomAndPan	กำหนดความสามารถในการย่อหรือขยาย
OnClick	mouse event
onmouseover	mouse event
onmouseout	mouse event
Transform	เคลื่อนย้ายหรือหมุนปรับอัตราส่วนของภาพ
xlink:href	เป็นการอ้างอิงถึง URL

ตารางที่ 2 -6 SVG Attribute

2.2.3 ภาษา จาวาสคริปต์ (JavaScript)

จาวาสคริปต์ เป็นภาษาสคริปต์ ที่สมบูรณ์ใกล้เคียงกับภาษาในการเขียนโปรแกรมปกติ จาวาสคริปต์มีคุณสมบัติพื้นฐานต่างๆ เช่นเดียวกับภาษาปกติ ไม่ว่าจะเป็นการกำหนดตัวแปร การกำหนดฟังก์ชันการควบคุมการทำงานของโปรแกรมสแตทเมนต์ และชุดของโอเปอเรเตอร์ต่างๆ นอกจากนี้จาวาสคริปต์ยังมีคุณสมบัติในการเขียนโปรแกรมแบบอ็อบเจกต์บางอย่างไว้ด้วย ดังเช่น จาวาสคริปต์สามารถเข้าไปใช้กับอ็อบเจกต์ ซึ่งปรากฏอยู่บนเว็บหรือจะแก้ไข Attribute รวมทั้งอีเวนต์ไว้ให้เรียกใช้งานด้วย จาวาสคริปต์นั้นช่วยให้เพจ HTML สามารถทำงานได้ ดังเช่นนักพัฒนาเว็บอาจจะสร้างให้เว็บเพจของตนสามารถตอบสนองต่อเหตุการณ์ หรืออีเวนต์

ของผู้ใช้ได้ไม่ว่าจะเป็นการคลิกเมาส์ หรือว่าการเรียกฟอร์มของ HTML ข้อดีของจาวาสคริปต์ที่สำคัญอีกประการหนึ่งคือ โครงสร้างของภาษากลายภาษางาวา ซึ่งง่ายต่อการเรียนรู้และใช้งาน

ความแตกต่างระหว่างภาษางาวาและจาวาสคริปต์ที่เห็นได้ชัดเจนคือ จาวาสคริปต์มีโครงสร้างของภาษาที่ไม่ซับซ้อนเหมือนกับจาวา นักพัฒนาเว็บที่คุ้นเคยกับ Tag HTML และเคยใช้งานภาษาในการเขียนภาษาซี สามารถเรียนรู้และใช้งานภาษาสคริปต์ได้ในเวลาไม่นาน นอกจากนั้นโปรแกรมที่เขียนด้วยภาษาสคริปต์นั้น ไม่จำเป็นที่จะต้องคอมไพล์เหมือนโปรแกรมจาวาทั่วไป ทั้งนี้เนื่องจากเป็นภาษาสคริปต์ โปรแกรมเว็บเบราว์เซอร์จึงมีตัวทำหน้าที่ปฏิบัติตามสคริปต์ที่เขียนไว้ทันที ในการพัฒนาแผนที่โดยใช้ภาษา GML เราสามารถใช้ภาษางาวาสคริปต์ในการเข้าถึง Element หรือ Attribute ของโครงสร้างภาษา SVG เพื่อสร้างส่วนของการปฏิสัมพันธ์กับผู้ใช้แผนที่เช่น การย่อ หรือ ขยายแผนที่ การเลื่อนแผนที่หรือ การนำข้อมูลรายละเอียดของสถานที่ต่างๆ ขึ้นมาแสดงโดยใช้ภาษา SVG

2.2.3.1 JavaScript คืออะไร

JavaScript เป็นภาษายุคใหม่สำหรับการเขียนโปรแกรมบนระบบอินเทอร์เน็ตที่กำลังได้รับความนิยมอย่างสูง เราสามารถเขียน โปรแกรม JavaScript เพิ่มเข้าไปในเว็บเพจเพื่อใช้ประโยชน์สำหรับงานด้านต่าง ๆ ทั้งการคำนวณ การแสดงผล การรับ-ส่งข้อมูล และที่สำคัญคือสามารถโต้ตอบกับผู้ใช้ได้อย่างทันทีทันใด นอกจากนี้ยังมีความสามารถด้านอื่น ๆ อีกหลายประการที่ช่วยสร้างความน่าสนใจให้กับเว็บเพจของเราได้อย่างดี

2.2.3.2 ลักษณะการทำงานของ JavaScript

JavaScript เป็นภาษาสคริปต์เชิงวัตถุ หรือเรียกว่า Object Oriented Programming ที่มีเป้าหมายในการออกแบบและพัฒนาโปรแกรมในระบบอินเทอร์เน็ต สำหรับผู้เขียนเอกสารด้วยภาษา HTML สามารถทำงานข้ามแพลตฟอร์มได้ ทำงานร่วมกับ ภาษา HTML และภาษางาวาได้ทั้งทางฝั่งไคลเอนต์ (Client) และ ทางฝั่งเซิร์ฟเวอร์ (Server) โดยมีลักษณะการทำงานดังนี้

1. Navigator JavaScript เป็น Client-Side JavaScript ซึ่งหมายถึง JavaScript ที่ถูกแปลทางฝั่งไคลเอนต์ (หมายถึงฝั่งเครื่องคอมพิวเตอร์ของผู้ใช้ไม่ว่าจะเป็นเครื่องพีซี เครื่องแมคอินทอช หรือ อื่นๆ) จึงมีความเหมาะสมต่อการใช้งานของผู้ใช้ทั่วไปเป็นส่วนใหญ่
2. LiveWire JavaScript เป็น Server-Side JavaScript ซึ่งหมายถึง JavaScript ที่ถูกแปลทางฝั่งเซิร์ฟเวอร์ (หมายถึงฝั่งเครื่อง คอมพิวเตอร์ของผู้ให้บริการเว็บ โดยอาจจะเป็นเครื่องของชั้น ซิลิคอนกราฟิก หรือ อื่น ๆ) สามารถใช้ได้เฉพาะกับ LiveWire ของเน็ตสเคป โดยตรง

ตัวแปร

ตัวแปร (Variable) หมายถึง ชื่อหรือสัญลักษณ์ที่ตั้งขึ้นสำหรับการเก็บค่าใดๆ ที่ไม่คงที่โดยการจองเนื้อที่ในหน่วยความจำของระบบเครื่องที่เก็บข้อมูลซึ่งสามารถอ้างอิงได้ มีขนาดขึ้นอยู่กับชนิดของข้อมูลและค่าของข้อมูล ซึ่งค่าในตัวแปรนี้สามารถเปลี่ยนแปลงได้ตามคำสั่งในการประมวลผล

การตั้งชื่อ

การตั้งชื่อ (Identifier or Name) เป็นชื่อที่ตั้งขึ้นมาเพื่อกำหนดค่าให้เป็นชื่อของโปรแกรมหลัก, ฟังก์ชัน, ตัวแปร, ค่าคงที่, คำสั่ง และคำสงวน โดยมีหลักการตั้งชื่อว่า

- เริ่มต้นด้วยตัวอักษรในภาษาอังกฤษ ตามด้วยตัวอักษรหรือตัวเลขใด ๆ ก็ได้
- ห้ามเว้นช่องว่าง
- ห้ามใช้สัญลักษณ์พิเศษ ยกเว้นขีดล่าง (_) และ ดอลลาร์ (\$)
- สำหรับความยาวของชื่อใน JavaScript จะมีความยาวเท่าใดก็ได้ แต่ที่นิยมใช้ ไม่เกิน 20 ตัวอักษร
- การตั้งชื่อมีข้อพึงระวังว่า จะต้องไม่ซ้ำกับคำสงวน (Reserve word) และตัวอักษรของชื่อจะจำแนกแตกต่างกันระหว่างอักษรตัวพิมพ์เล็กกับอักษรตัวพิมพ์ใหญ่
- ควรจะตั้งชื่อโดยให้ชื่อนั้นมีสื่อความหมายให้เข้ากับข้อมูล สามารถอ่านและเข้าใจได้

คำสงวน

คำสงวน (Reserve word) เป็นคำที่มีความหมายเฉพาะตัวในภาษา JavaScript สงวนไม่ให้มีการตั้งชื่อซ้ำกับชื่อโปรแกรม, ฟังก์ชัน, ตัวแปร, ค่าคงที่ และคำสั่ง คำสงวน สามารถเรียกใช้ได้ทันทีโดยไม่ต้องมากำหนดความหมายใหม่แต่อย่างใด

ชนิดข้อมูลของตัวแปร

ชนิดของข้อมูลของตัวแปร (Data Type) เป็นการกำหนดประเภทค่าของข้อมูลให้กับตัวแปร เพื่อให้เหมาะสมกับการอ้างอิงข้อมูลจากตัวแปรในการใช้งาน ชนิดข้อมูลของตัวแปรนั้นมีอยู่ด้วยกัน 4 ชนิด ได้แก่

- Number หมายถึง ข้อมูลชนิดตัวเลข ประกอบด้วย เลขจำนวนเต็ม (Integer) และเลขจำนวนจริง (Floating)
- Logical หมายถึง ข้อมูลทางตรรกะ มี 2 สถานะ คือ จริง (True) และเท็จ (False)
- string หมายถึง ข้อมูลที่เป็นข้อความ ซึ่งจะต้องกำหนดไว้ในเครื่องหมายคำพูด ("...")
- Null หมายถึง ไม่มีค่าข้อมูลใดๆ ซึ่งค่า null ใช้สำหรับการยกเลิกพื้นที่เก็บค่าของตัวแปรออกจากหน่วยความจำ

การประกาศตัวแปร

การประกาศตัวแปร (Declarations) เป็นการกำหนดชื่อและชนิดข้อมูลให้กับตัวแปรเพื่อนำไปใช้ในโปรแกรม โดยการตั้งชื่อจะต้องคำนึงถึงค่าของข้อมูลและ ชนิดของข้อมูลที่อ้างอิง นอกจากนี้การตั้งชื่อควรให้สื่อความหมายของข้อมูล และอักษรของชื่อจะจำแนกแตกต่างกันระหว่างอักษรตัวพิมพ์เล็กกับอักษรตัวพิมพ์ใหญ่

รูปแบบ

Var ชื่อตัวแปร; เป็นรูปแบบการประกาศตัวแปรปกติ หรือ

Var ชื่อตัวแปร = ข้อมูล; เป็นรูปแบบการกำหนดค่าเริ่มต้น

ในกรณีที่ต้องการกำหนดตัวแปรหลายตัวในบรรทัดเดียวกันให้ใช้เครื่องหมาย คอมม่า (,) คั่นระหว่างชื่อตัวแปรและปิดท้ายด้วยเครื่องหมายเซมิโคลอน (;)

การกำหนดค่าให้กับตัวแปร

รูปแบบ

ชื่อตัวแปร = ค่าของข้อมูล

โดยที่

ค่าของข้อมูล ได้แก่

1. ข้อมูลที่เป็นตัวเลข โดยกำหนดตัวเลขไปได้เลย เช่น num = 500;
2. ข้อมูลในทางตรรกะ ได้แก่ จริง (True) หรือ เท็จ (False) เช่น test = True;
3. ข้อมูลสตริง ให้กำหนดอยู่ในเครื่องหมายคำพูด ("...") เช่น name = "Adisak";

ตัวแปรมี 2 จำพวก หากเรากำหนดชื่อตัวแปรไว้ที่โปรแกรมหลักโดยไม่ได้อยู่ภายในขอบเขตฟังก์ชันใด ๆ เราเรียกว่าเป็นตัวแปรแบบโกลบอล (global) ตัวแปรจำพวกนี้จะมีค่าคงอยู่ในหน่วยความจำตลอดการทำงานของโปรแกรม ทำให้สามารถเรียกใช้ได้จากทุก ๆ ส่วนของโปรแกรม รวมถึงภายในฟังก์ชันต่าง ๆ ด้วย

แต่ถ้ากำหนดตัวแปรไว้ในขอบเขตฟังก์ชันใด ๆ เราจะเรียกว่าเป็นตัวแปรแบบ โลคัล (Local) เพราะจะเป็นตัวแปรที่มีค่าคงอยู่ และสามารถเรียกใช้ได้เฉพาะ ภายในขอบเขตของฟังก์ชันนั้น ๆ เท่านั้น

ตัวแปรแบบอาร์เรย์

ตัวแปรแบบอาร์เรย์ (Array) หมายถึงตัวแปรซึ่งมีค่าได้หลายค่าโดยใช้ชื่ออ้างอิงเพียงชื่อเดียว ด้วยการให้หมายเลขลำดับเป็นตัวจำแนกความแตกต่างของค่าตัวแปรแต่ละตัว ถ้าเราจะเรียกตัวแปรชนิดนี้ว่า "ตัวแปรชุด" ก็เห็นจะไม่ผิดนัก ตัวแปรชนิดนี้มีประโยชน์มาก ลองคิดถึงค่าข้อมูลจำนวน 100 ค่า ที่ต้องการเก็บไว้ในตัวแปรจำนวน 100 ตัว อาจทำให้ต้องกำหนดตัวแปรที่แตกต่างกันมากถึง 100 ชื่อ กรณีอย่างนี้ควรจะทำอย่างไรดี

แต่ด้วยการใช้คุณสมบัติอาร์เรย์ เราสามารถนำตัวแปรหลาย ๆ ตัวมาอยู่รวมเป็นชุดเดียวกันได้ และสามารถเรียกใช้ตัวแปรทั้งหมดโดยระบุผ่านชื่อเพียงชื่อเดียวเท่านั้น ด้วยการระบุหมายเลขลำดับ หรือ ดัชนี(index) กำกับตามหลังชื่อตัวแปร ตัวแปรเพียงชื่อเดียวจึงมีความสามารถเทียบได้กับตัวแปรนับร้อยตัว พันตัว (ตัวที่ 1) ในตัวแปรแบบอาร์เรย์มีดัชนีเป็น 0 ส่วนตัวแปรต่อ ๆ ไปก็จะมีดัชนีเป็น 1,2,3,... ไปตามลำดับ เมื่อต้องการระบุชื่อตัวแปรแบบอาร์เรย์แต่ละตัว ก็จะใช้รูปแบบ name[0], name[1],... เรียงต่อกันไปเรื่อยๆ เราสามารถสร้างตัวแปรอาร์เรย์ใหม่ด้วย myArray = new Array() ดังนี้

```
myArray[0] = 17;
myArray[1] = "Nun";
myArray[2] = "Stop";
```

ค่าคงที่

ค่าคงที่ (Literal หรือ Constant) หมายถึง ค่าของข้อมูลที่เมื่อกำหนดแล้วจะทำการเปลี่ยนแปลงค่าเป็นอย่างอื่นไม่ได้ ชนิดข้อมูลของค่าคงที่ได้แก่

- เลขจำนวนเต็ม (Integer) เป็นตัวเลขที่ไม่มีเศษทศนิยม สามารถเขียนให้อยู่ในแบบเลขฐานสิบ (0-9), เลขฐานสิบหก (0-9, A-F) หรือ เลขฐานแปด (0-7) โดยการเขียนเลขฐานแปดให้ นำหน้าด้วย O (Octonary) ส่วนการเขียนเลขฐานสิบหกให้นำหน้าด้วย Ox หรือ OX (Hexadecimal)
- เลขจำนวนจริง (Floating) ใช้รูปแบบการเขียนโดยประกอบไปด้วยตัวเลข จุดทศนิยม และตัวเลขยกกำลัง E (Exponential) เช่น
 - 5.00E2 จะหมายถึงค่า 5.00 คูณด้วย 10 ยกกำลัง 2 จะมีค่าเป็น 500
 - 3.141E5 จะหมายถึงค่า 3.141 คูณด้วย 10 ยกกำลัง 5 จะมีค่าเป็น 314,1000
- ค่าบูลีน (Boolean) เป็นค่าคงที่เชิงตรรกะ คือมีค่าเป็น จริง(True) และ เท็จ(False) เท่านั้น
- ข้อความสตริง (String) เป็นค่าคงที่แบบข้อความที่อยู่ภายในเครื่องหมายคำพูด ("..." หรือ '...') เช่น "บริษัท เอ็กซ์ทริม จำกัด", 'นางนฤมล เวชตระกูล'

นิพจน์

นิพจน์ (Expression) เป็นข้อความที่ใช้กำหนดค่าของข้อมูล เช่น การบวกตัวเลข การเปรียบเทียบข้อมูล โดยการกำหนดชื่อของตัวแปร ตามด้วยเครื่องหมายที่ต้องการกระทำ (Operations) ต่อข้อมูลเป็นผลให้เกิดค่าข้อมูลใหม่ค่าหนึ่งให้กับตัวแปรเพื่อนำไปใช้งาน นิพจน์ JavaScript มีด้วยกัน 3 ชนิดดังนี้

1. นิพจน์คณิตศาสตร์ (Arithmetic) เป็นนิพจน์ที่ใช้เครื่องหมายทางคณิตศาสตร์เป็นตัวกระทำ ผลลัพธ์ที่ได้จะมีค่าเป็นตัวเลขให้กับตัวแปร เช่น ให้ตัวแปร num เก็บตัวเลข 5000 จะเขียนได้ดังนี้ `num = 5000;`
2. นิพจน์ตรรกะ (Logical) เป็นนิพจน์ในการเปรียบเทียบข้อมูลโดยใช้เครื่องหมายในการเปรียบเทียบเพื่อตรวจสอบข้อมูลในการเปรียบเทียบว่าจริงหรือเท็จ เช่น กำหนดให้ `a = 50; b = 70; c = b > a;` ผลลัพธ์ที่ได้คือ c จะมีค่าเป็นจริง (True)
3. นิพจน์ข้อความ (String) เป็นนิพจน์เกี่ยวกับการกำหนดข้อความ การเชื่อมต่อข้อความ ใช้ประมวลผลข้อความในลักษณะต่าง ผลลัพธ์ที่ได้จึงมีค่าเป็นตัวอักษรหรือข้อความเสมอ เช่น ให้ตัวแปร name เก็บชื่อ Adisak จะเขียนได้ดังนี้ `name = "Adisak";`

ตัวดำเนินการ

ตัวดำเนินการ (Operator) หมายถึง เครื่องหมายกำหนดกรรมวิธีทางคณิตศาสตร์, พีชคณิต, บูลีน, การเปรียบเทียบ ระหว่างข้อมูล 2 ตัว ซึ่งเรียกว่า Operand โดยอาจมีค่าเป็นตัวเลข ข้อความ ค่าคงที่ หรือตัวแปรต่าง ๆ

ชนิดของตัวดำเนินการ

ตัวดำเนินการคณิตศาสตร์ (Arithmetic operator) หมายถึง ใช้สำหรับคำนวณ Operand ที่เป็นค่าคงที่หรือตัวแปรก็ได้ โดยให้ค่าผลลัพธ์เป็นตัวเลขค่าเดียว Operator เชิงคณิตศาสตร์ที่คนส่วนใหญ่รู้จักคุ้นเคยกันมากที่สุดได้แก่ `+`, `-`, `*`, `/`, `%`, `++`, `--` และ `(-)` เป็นต้น

รูปแบบ

เช่น `x = 20 % 3;` ผลลัพธ์คือ x จะมีค่าเป็น 2 ,

ถ้า `x = -100` ดังนั้น `-x` จะมีค่าเท่ากับ 100 เป็นต้น

ตัวดำเนินการเชิงเปรียบเทียบ (Comparison operator) หมายถึง เครื่องหมายในการเปรียบเทียบข้อมูล ผลลัพธ์ที่ได้จะมีค่าตรรกะบูลีนเป็น จริง (True) และ เท็จ (False) ได้แก่ `==`, `!=`, `>`, `>=`, `<` และ `<=` เป็นต้น

ตัวดำเนินการเชิงตรรกะ (Logical operator) เป็นเครื่องหมายที่ให้ค่าจริง (True) และ เท็จ (False) ในการเปรียบเทียบ ประกอบด้วยเครื่องหมาย `&&` (AND) , `||` (OR) , `!` (NOT)

ตัวดำเนินการเชิงข้อความ (String operator) เป็นการเชื่อมต่อข้อความเข้าด้วยกัน (concatenation) โดยใช้เครื่องหมายบวก (+) เป็นตัวกระทำเช่น

`Name = "Bodin"; Say = "Hey "+Name;` ผลลัพธ์ที่ได้ Say จะมีข้อความเป็น Hey Bodin

การสร้างตัวแปรอาร์เรย์

ก่อนที่จะมีการใช้งานอาร์เรย์นั้น เรา จะต้องทำการประกาศตัวแปรที่มีลักษณะเป็นอาร์เรย์เสียก่อน เพื่อให้โปรแกรมรู้จักชื่อของตัวแปรที่จะกำหนดเป็นอาร์เรย์ พร้อมถึงการกำหนดขนาดของพื้นที่ในหน่วยความจำสำหรับเก็บค่าของข้อมูล รายละเอียดของการประกาศสร้างตัวแปรอาร์เรย์ มีดังต่อไปนี้

การประกาศตัวแปรอาร์เรย์ ชื่อตัวแปรอาร์เรย์ = new Array (จำนวนสมาชิก);
รายละเอียดมีดังนี้ จำนวนสมาชิก หมายถึง การกำหนดการจองพื้นที่ในหน่วยความจำ ให้กับตัวแปรเพื่อรองรับข้อมูลที่กำหนด โดยปกติจะกำหนดหรือไม่ก็ได้ เพราะอาร์เรย์ของจาวาสคริปต์มีความยืดหยุ่นมากสำหรับการรับจำนวนสมาชิก การกำหนดค่าให้กับตัวแปรอาร์เรย์ ตัวแปร [Index] = ข้อมูล; รายละเอียดมีดังนี้ Index หมายถึง หมายเลขลำดับของพื้นที่ที่เก็บข้อมูลโดยเริ่มนับตั้งแต่ 0, 1, 2 ... เป็นต้นไป

ฟังก์ชันและเมธอด

ฟังก์ชัน (Function) โดยเนื้อแท้แล้วก็คือโปรแกรมย่อย เป็นส่วนของโปรแกรมที่ถูกกำหนดให้ทำงานใดงานหนึ่งจนสำเร็จ มีประโยชน์ตรงที่ช่วยเหลือการทำงาน หรือตอบสนองการทำงานต่อโปรแกรมหลัก เมื่อมีการเรียกใช้งาน และมีประโยชน์ทำให้โครงสร้างของโปรแกรม มีขนาดเล็กลง เมื่อมีการเรียกใช้งานเดิมซ้ำๆ หลายๆ ครั้ง แทนที่จะเขียนคำสั่งเดิมเพิ่มขึ้นใหม่ซ้ำๆ หลายครั้ง ทำให้ดูสั้นเปลืองเนื้อที่ ช้าช้อนและเสียเวลาการทำงาน นอกจากนี้ฟังก์ชันยังทำให้โปรแกรมอ่านได้ง่ายขึ้น สะดวกในการหาจุดที่ผิดและง่ายต่อการปรับปรุงแก้ไขพัฒนาโปรแกรมให้ดียิ่งขึ้น ชนิดของฟังก์ชัน

ฟังก์ชันใน ภาษาJavaScript มีอยู่ด้วยกัน 2 แบบ คือ

- ฟังก์ชันมาตรฐาน (Standard Function) เป็นแบบชื่อของฟังก์ชันที่มีอยู่แล้วในภาษา JavaScript เราสามารถนำไปใช้งานได้ทันที
- ฟังก์ชันสร้างขึ้นเอง (User-defined Function) เป็นแบบชื่อของฟังก์ชันที่ผู้ใช้สร้างขึ้นมาใช้เอง เพื่อกำหนดให้ทำงานใดงานหนึ่งจนสำเร็จ เราอาจจะเรียกฟังก์ชันสร้างขึ้นเองนี้สั้นๆ ว่าฟังก์ชัน (Function) ก็ได้

การเรียกใช้ฟังก์ชัน

เป็นการกำหนดทิศทางการทำงานของคำสั่ง หรือ กำหนดส่วนของโปรแกรมให้ทำงานใดงานหนึ่งจนสำเร็จ โดยอ้างอิงชื่อฟังก์ชันของการทำงานที่ต้องการผลของการเรียกใช้ฟังก์ชันจะมีการส่งค่าคืนกลับไปยังโปรแกรมส่วนที่เรียก โดยใช้ชื่อฟังก์ชันเป็นตัวเก็บค่าเปรียบเสมือนหนึ่งกับเป็นตัวแปร เมื่อต้องการใช้งานก็ให้พิมพ์ชื่อฟังก์ชันนี้ลงไปตรงๆ แต่จะต้องจำให้แม่นว่าต้องพิมพ์ให้เหมือนทั้งชื่อและตัวอักษรตัวพิมพ์ใหญ่-เล็ก มีรูปแบบการเรียกใช้ดังนี้

รูปแบบ

ตัวแปร = ชื่อฟังก์ชัน();

โดยกำหนดให้

ตัวแปร ทำหน้าที่เก็บผลลัพธ์ที่ได้จากการอ้างอิงเรียกใช้ฟังก์ชัน เพื่อให้ทำงานใดงานหนึ่งจนสำเร็จ การสร้างฟังก์ชันขึ้นใช้เอง

การสร้างฟังก์ชันขึ้นใช้เอง (User-defined Function) เป็นแบบชื่อของฟังก์ชันที่ผู้ใช้สร้างขึ้นมาเพื่อกำหนดให้ทำงานใดงานหนึ่งจนสำเร็จ การสร้างฟังก์ชันนั้นจะขึ้นต้นด้วยคำว่า function ตามด้วยชื่อของฟังก์ชันที่ต้องการ มีรายละเอียดดังนี้

รูปแบบ

```
function ชื่อฟังก์ชัน (พารามิเตอร์1, พารามิเตอร์2, ...)
{
    คำสั่ง
    .....
}
```

โดยกำหนดให้

- ชื่อฟังก์ชัน (function name) - หมายถึง ชื่อของฟังก์ชันที่สร้างขึ้น ที่ไม่ไปซ้ำกับชื่อของฟังก์ชันอื่น
- พารามิเตอร์ (parameter) - หมายถึง ค่าของข้อมูลหรือตัวแปรที่อ้างอิง (argument) แล้วส่งผ่านไปยังฟังก์ชัน ต้องระบุอยู่ในเครื่องหมายวงเล็บเท่านั้น โดยจะมีพารามิเตอร์เพียงตัวเดียว, หลายตัว หรือไม่มีเลยก็ได้ กรณีที่มีพารามิเตอร์หลาย ๆ ตัว แต่ละตัวจะต้องเขียนแยกออกจากกันด้วยเครื่องหมายจุลภาค (,)
- คำสั่ง (statements) - หมายถึง คำสั่งต่าง ๆ ที่ประกอบขึ้นเพื่อให้ดำเนินงานภายในฟังก์ชัน ต้องกำหนดคำสั่งต่าง ๆ ภายในเครื่องหมายวงเล็บปีกกา {.....}

การวางตำแหน่งฟังก์ชัน

สำหรับการวางตำแหน่งฟังก์ชันในภาษาJavaScript ก็มีลักษณะเช่นเดียวกับการวางตำแหน่งสคริปต์ นั่นคือจะวางไว้ในส่วนของ <HEAD> หรือวางไว้ในส่วนของ <BODY>อย่างไรก็ขึ้นอยู่กับว่าต้องการให้ฟังก์ชันนั้นถูกโหลดใช้งานก่อนหรือหลังตามลำดับการเรียกใช้งานอย่างไร ในกรณีที่ฟังก์ชันนั้นมีการถูกเรียกใช้บ่อยครั้งจากส่วนอื่น ๆ ของโปรแกรม ทางคณะผู้จัดทำแนะนำว่า ควรจะกำหนดฟังก์ชันไว้ในส่วนของ <HEAD> เพราะเมื่อมีการเรียกใช้โหลดเว็บเพจขึ้นมา ฟังก์ชันต่าง ๆ ที่กำหนดในส่วน <HEAD> จะถูกโหลดเข้ามาเก็บไว้ในหน่วยความจำก่อนเป็นอันดับแรก ทำให้เราสามารถเรียกใช้ฟังก์ชันจากตำแหน่งใดๆ บนเอกสาร HTML หรือบนขอบเขต

<SCRIPT> "ได้อย่างต่อเนื่อง และนอกจากนี้ฟังก์ชันยังสามารถเรียกใช้ฟังก์ชันอื่นๆ ที่กำหนดในส่วน <HEAD> ทำงานร่วมกันได้อีกด้วย

รูปแบบ

```
<html>

<head><title> ชื่อความแถบเรื่อง </title>

<script Language = "JavaScript">
<!-- ชื่อความหมายเหตุ ...
function ชื่อฟังก์ชัน (พารามิเตอร์1, พารามิเตอร์2, ...)
{ ชื่อคำสั่ง ;
.....
}
กลุ่มโค้ดคำสั่งของภาษาจาวาสคริปต์
// ชื่อความหมายเหตุ -->
</script>
</head>

<body>
ชื่อความเอกสาร html
.....

<script Language = "JavaScript">
กลุ่มโค้ดคำสั่งของภาษาจาวาสคริปต์
.....
</script>
</body>
</html>
```

โดยกำหนดให้

กลุ่มโค้ดคำสั่งของภาษาจาวาสคริปต์ หมายถึง คำสั่งใด ๆ หรือ ฟังก์ชัน หรือ การเรียกใช้ฟังก์ชันที่ต้องการใช้งาน

2.2.4 Ajax (Asynchronous JavaScript and XML)

ปัจจุบันนี้ ลักษณะการทำงานแบบ Client - Server เริ่มถูกนำมาใช้งานอย่างแพร่หลายในลักษณะการติดต่อสื่อสารผ่านทาง Web browser ซึ่งการทำงานแบบนี้ จะมีการทำงานโดย Client จะร้องขอและต้องการข้อมูลบางอย่างจาก Server ดังนั้นการโหลดและการรีเฟรชหน้าจอ เป็นสิ่งที่หลีกเลี่ยงไม่ได้ จึงเป็นผลให้การทำงานของฝั่ง Client นี้ทำให้ผู้ใช้ต้องหยุดรอการโหลดและการรีเฟรชหน้าจอ ซึ่งถือว่าเป็นการทำงานที่ไม่มีประสิทธิภาพ

2.2.4.1 Ajax คืออะไร

Ajax ไม่ใช่ชื่อของการเขียนโปรแกรม หรือเป็นชื่อของภาษาที่ใช้ในการโปรแกรม แต่เป็นชุดของเทคโนโลยีต่างๆ Ajax ย่อมาจาก Asynchronous JavaScript And XML; ซึ่งหมายถึงการทำงานร่วมกันของ JavaScript และ XML แบบ Asynchronous มีหลักการทำงาน 2 ประเด็น คือ การ update หน้าจอแบบบางส่วน และการติดต่อสื่อสารกับ Server โดยใช้หลักการ Asynchronous ทำให้ผู้ใช้ไม่ต้องหยุดการทำงาน เพื่อรอการประมวลผลจาก Server รวมถึงการโหลดและการรีเฟรชหน้าจอ ของบราวเซอร์ทางฝั่ง Client มีการใช้ Ajax โดยการเพิ่มเลเยอร์ระหว่าง user browser กับ server ทำให้ผู้ใช้สามารถทำงานได้โดยไม่ต้องรอให้ Client ติดต่อไปยัง Server รวมถึงการโหลดและการรีเฟรชหน้าจอทั้งหมดด้วย ดังนั้นผู้ใช้สามารถใช้งาน application ได้อย่างมีประสิทธิภาพมากขึ้น

สรุปก็คือ Ajax จึงไม่ใช่เทคโนโลยีในตัวของมันเองและ ไม่ใช่เทคโนโลยีล่าสุดของการพัฒนาเว็บแอปพลิเคชัน แต่ Ajax เป็นการนำเทคโนโลยีหลายๆ ตัวมารวมกันเช่น JavaScript, DHTML, XML, Css, DOM และ XMLHttpRequest มันประกอบไปด้วยเทคโนโลยีที่เราคุ้นเคยกันคืออยู่แล้ว แต่ถูกจับมารวบอยู่ด้วยกัน Ajax ในวันนี้มีแนวโน้มที่จะเติบโตอย่างรวดเร็วในอนาคตอันใกล้ เพราะแม้แต่ Google ที่ได้รับการยอมรับว่ามีการพัฒนาเรื่องของเทคโนโลยีทางเว็บก้าวหน้าอย่างมากยังนำ Ajax มาใช้ในแอปพลิเคชันของ

Ajax engine ทำหน้าที่เป็นตัวกลางระหว่าง client และ server ฉะนั้นเมื่อ client มี request แทนที่จะส่ง HTTP request ไปยัง server โดยตรง client จะส่ง JavaScript call ไปยัง Ajax engine เพื่อโหลดข้อมูลที่ user ต้องการ และหาก Ajax engine ต้องการข้อมูลเพิ่มเติมในการตอบสนองต่อ user Ajax engine จะส่ง request ไปยัง server โดยใช้ XML เทคโนโลยีต่าง ๆ ที่เป็นส่วนประกอบของ Ajax ซึ่งได้แก่

- HTML/XHTML เป็นภาษาในการจัดแสดงข้อมูล
- CSS เป็นรูปแบบการจัดแต่ง XHTML
- Document Object Model (DOM) สำหรับ dynamic display and interaction

- XML เป็นรูปแบบการแลกเปลี่ยนข้อมูล
- XSLT สำหรับ แปลง XML เป็น XHTML
- XMLHttpRequest สำหรับ asynchronous data retrieval
- JavaScript เป็นภาษาในการใช้งาน Ajax engine

โดยส่วนประกอบจำเป็นขั้นพื้นฐานที่ขาดไม่ได้ใน Ajax ได้แก่ HTML/XHTML DOM และ JavaScript เพราะ XHTML

เนื่องจากแอปพลิเคชันที่ใช้งานในปัจจุบันนี้ มีหลักการที่ทำงานแล้วเกิดการสูญเสียเวลาและทรัพยากรของผู้ใช้ในการรอคอยการทำงานต่างๆ ทำให้ผู้ใช้ต้องหยุดคอย ดังนั้นการทำงานของผู้ใช้จึงเป็นไปอย่างไม่ต่อเนื่อง ซึ่งหลักการดังกล่าวคือ

■ "Click, wait, and refresh" user interaction paradigm

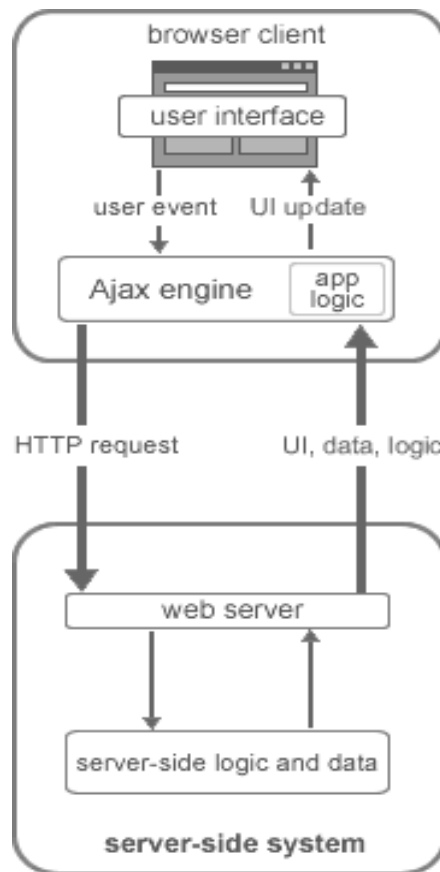
การที่เบราว์เซอร์ตอบสนองต่อการทำงานของผู้ใช้ โดยจะทิ้งหน้าเว็บที่แสดงอยู่ในขณะนั้น แล้วไปทำการส่ง HTTP request กลับไปยัง server แทน ซึ่งทำให้ผู้ใช้ไม่สามารถทำอะไรได้เลยในขณะนั้น นอกจากการรอคอย เมื่อ server ทำการประมวลผลเสร็จก็จะส่งหน้า HTML กลับมายังเบราว์เซอร์ ต่อจากนั้นเบราว์เซอร์ก็จะรีเฟรชและแสดงหน้า HTML หน้าใหม่ และนี่เองที่ทำให้ผู้ใช้สามารถใช้งานต่อไปได้ จะเห็นว่า ผู้ใช้มีช่วงเวลาของการหยุดรอคอยเป็นเวลานานสำหรับการประมวลผลของ Server และการรีเฟรชหน้า HTML ใหม่ทั้งหน้า ซึ่งเป็นสิ่งที่ไม่มีประสิทธิภาพในเชิง Dynamic ของการทำงานบนเว็บแอปพลิเคชัน

■ Synchronous "request/response" communication mode

การที่เบราว์เซอร์เริ่มทำการร้องขอข้อมูล และ server ก็ตอบสนองเฉพาะการร้องขอที่เบราว์เซอร์ร้องขอมา server จะไม่สามารถส่งข้อมูลได้ถ้าเบราว์เซอร์ไม่ได้ร้องขอข้อมูลในขณะนั้น ซึ่งถือว่าการติดต่อสื่อสารเป็นแบบทิศทางเดียว วงจรการ request/response แบบ synchronous คือ การทำงานแบบประสานจังหวะระหว่างเบราว์เซอร์กับ Server ทำให้เกิดความล่าช้าในการทำงาน ทำให้ผู้ใช้ทำอะไรไม่ได้อีก นอกจากการคอยการตอบสนองกลับมาจาก server เมื่อ server ประมวลผลเสร็จ

2.2.4.2 สถาปัตยกรรมของ Ajax

มุมมองของโครงสร้างทาง Software ของ Ajax ต่างจากเว็บแอปพลิเคชันในทุกวันนี้ เนื่องจากการเพิ่ม engine ทางฝั่ง client

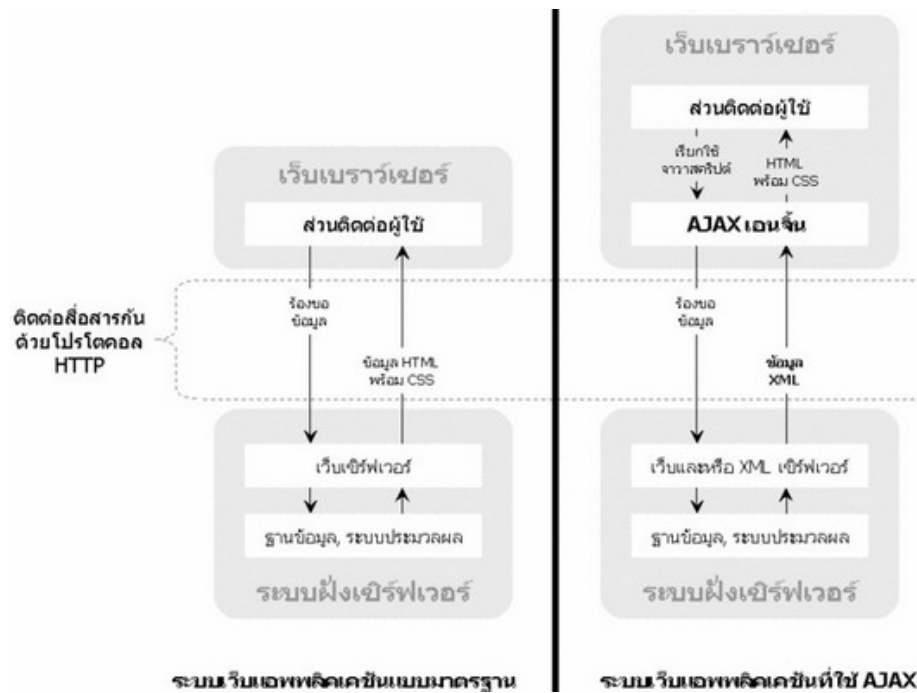


รูปที่ 2-8 Ajax architecture

จากรูป Ajax engine นี้ อยู่ระหว่าง User Interface กับ server ซึ่งจะมองว่าเป็นการทำงานที่ Client การทำงานต่างๆของผู้ใช้ โปรแกรมจะไปเรียก Ajax engine ตัวนี้ขึ้นมา แทนที่การร้องขอ หน้าเว็บจาก server โดยตรง และจะใช้โครงสร้างข้อมูลแบบ XML ในการขนย้ายข้อมูลระหว่าง server กับ Ajax engine เมื่อเบราว์เซอร์ทำการร้องขอข้อมูลจาก server นอกจากนี้ Ajax engine ไม่ต้องการติดตั้ง ไม่ใช่ plug-in และไม่สามารถ download ได้ เพราะ Ajax เป็นแนวคิดในการแก้ปัญหาการหยุดชะงักการ ทำงานของผู้ใช้

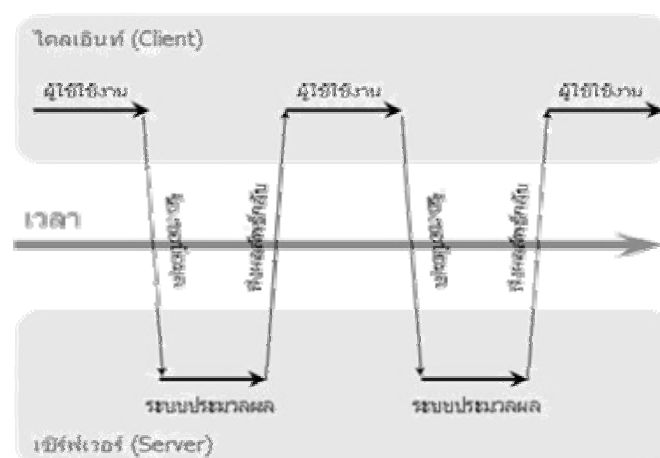
2.2.4.3 หลักการและการทำงานของ Ajax

Ajax (Asynchronous JavaScript and XML) คือเทคโนโลยีที่รวมเอาความสามารถของ JavaScript, XML, CSS และ XHTML เอาไว้ด้วยกัน Ajax เป็นการประยุกต์เอาเทคโนโลยีเก่ามาผสมผสานจนได้เทคโนโลยีใหม่ที่นำศึกษาและนำมาใช้งาน แต่ก่อนอื่นมาทำความเข้าใจ หลักการทำงานของ Web กันก่อน

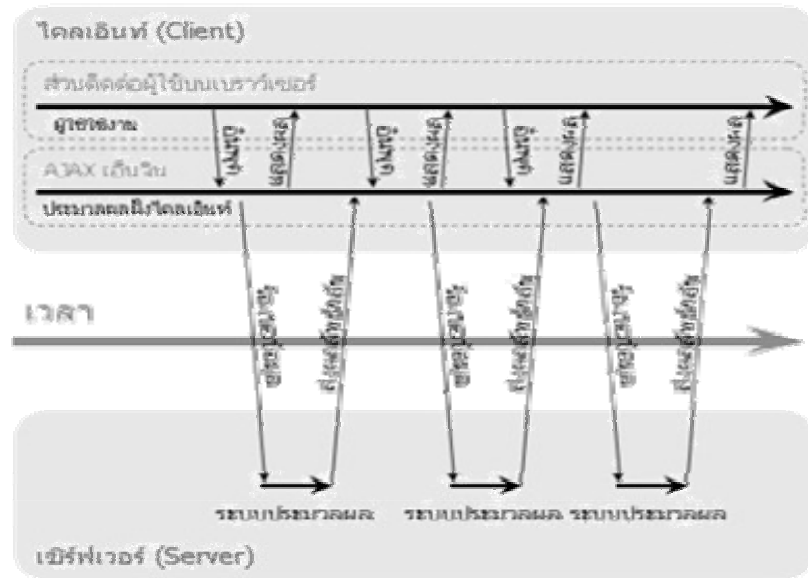


รูปที่ 2-9 เปรียบเทียบระบบการทำงานของเว็บแบบมาตรฐานและแบบใช้ Ajax

ตามปกติเมื่อเราเปิด Web Browser และพิมพ์ URL ของเว็บเพจที่ต้องการ เราจะเรียกผู้ใช้งาน client-side browser ก็จะส่งค่าไปยัง server เพื่อขอเปิดหน้า url ที่เราพิมพ์ลงไป เช่น <http://www.google.co.th> และเมื่อทาง server ได้รับค่าที่ส่งมาก็จะส่งหน้าเว็บเพจกลับมาให้ โดยเราจะเรียก server ว่าผู้ให้บริการหรือ server side เมื่อฝั่งผู้ใช้ได้รับข้อมูลจาก server ที่ส่งมาให้ browser ก็จะนำข้อมูลนั้นขึ้นหน้าจอ จากนั้นเมื่อเราคลิกเว็บหน้าอื่นๆก็จะเริ่มขั้นตอนทั้งหมดใหม่อีกครั้ง วิธีการนี้หน้าจอ webpage จะต้องมีการ refresh ใหม่ และการรับผลที่ส่งกลับมาก็จะเป็นการส่งมาแบบทั้งหน้า webpage แบบเต็มๆ ทำให้กิน bandwidth มาก



รูปที่ 2-10 ระบบเว็บแอปพลิเคชันแบบมาตรฐาน (Synchronize)



รูปที่ 2-11 ระบบเว็บแอปพลิเคชันแบบที่ใช้ Ajax (Asynchronous)

แล้ว Ajax คือว่าตรงที่การทำงานของ Ajax นั้นจะส่งเฉพาะข้อมูลที่ต้องการไปยัง server และส่งกลับมาเฉพาะข้อมูลที่ต้องการไม่ใช่การส่งทั้งหน้า webpage ใหม่ โดย Ajax อาศัย object ที่ชื่อ XMLHttpRequest เมื่อผู้ใช้เปิดหน้าเว็บแล้วมีการส่งข้อมูล Ajax ก็จะให้ XMLHttpRequest ส่งค่าไปให้ server แล้วให้ server จัดการข้อมูลนั้นตามเงื่อนไขแล้วส่งข้อมูลนั้นกลับมาในรูปแบบ XML ซึ่งก็จะใช้ JavaScript เป็นตัวจัดการข้อมูลที่ได้รับให้แสดงผลได้อย่างถูกต้องในหน้าเว็บเพจเดิม

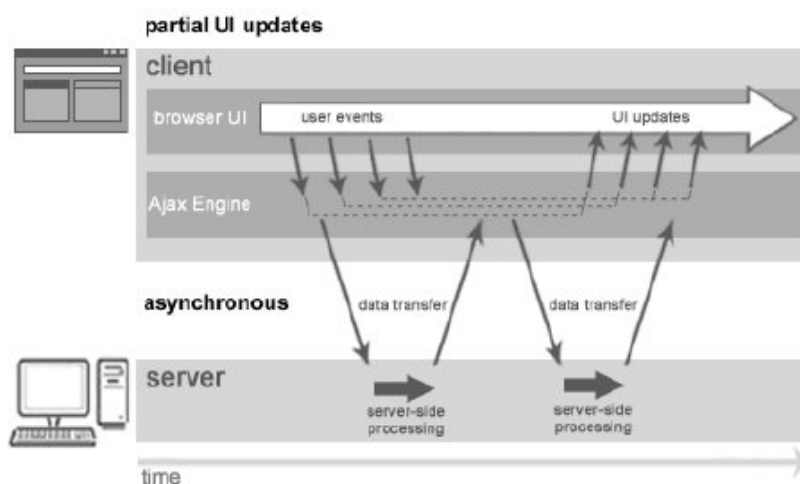
AJAX จะช่วยลดการติดต่อระหว่าง Client กับ Server โดยในการโหลดหน้าเว็บนั้น browser จะโหลดข้อมูลจาก AJAX engine แทนการร้องขอข้อมูลจาก server โดยตรง ดังนั้น Ajax จะทำหน้าที่ทั้งการ render ส่วนติดต่อกับผู้ใช้และติดต่อไปยัง server แล้ว AJAX engine อนุญาตให้การกระทำต่างๆ ใน web application เป็นแบบ Asynchronous ก็คือความเป็นอิสระในการติดต่อไปยัง server นั้นเอง ดังนั้นผู้ใช้จะไม่พบกับ browser หน้าขาวๆ อีกต่อไป และไม่ต้องรอการโหลดข้อมูลต่างๆ จาก server

■ **"Partial screen update" replaces the "click, wait, and refresh" user interaction model**

การ Update หน้าจอบางส่วน แทนที่การ "click, wait, and refresh" ระหว่างที่เกิด การทำงานแบบการติดต่อสื่อสารของผู้ใช้ user interface ที่ต้องนำมาแสดงซ้ำในหน้าเว็บที่ร้องขอไปยัง server จะถูกจัดเป็นข้อมูลใหม่เมื่อถูก update แล้ว การหยุดชะงักของ user interface จึงไม่เกิดขึ้น เพราะหน้าเว็บนั้นยังคงถูกแสดงอยู่และสามารถใช้งานได้ โดยปราศจากการหยุดชะงักการทำงานของ ผู้ใช้ การ update หน้าเว็บบางส่วนสามารถทำให้หน้าเว็บทำงานต่อไปได้ ถึงแม้จะไม่ทั้งหมด แต่อย่างน้อยก็ทำให้การทำงานไม่จำเป็นต้องหยุดชะงักเลย

■ Asynchronous communication replaces "synchronous request/response model"

การติดต่อแบบ Asynchronous เข้ามาแทนที่การ "synchronous request/response model" สำหรับ Ajax การ request/response จะทำแบบ asynchronous ซึ่งคือการติดต่อสื่อสารกับ server แบบอิสระโดยทำการลดการติดต่อระหว่างเบราว์เซอร์ กับ server ผลที่ได้ก็คือผู้ใช้สามารถใช้งานเว็บแอปพลิเคชันได้ในขณะที่ client ทำการร้องขอข้อมูลจาก server อยู่เบื้องหลัง(การทำงานแบบพร้อมกันแต่มองเป็น 2 ฟาก เช่นหน้าร้านกับหลังร้าน) เมื่อข้อมูลเดินทางมาถึงเบราว์เซอร์ ก็จะ update หน้า user interface ที่ต้องการข้อมูลใหม่ ส่วนหน้า user interface ที่ไม่ต้องการ update ก็จะแสดงส่วนนั้นต่อไป



รูปที่ 2-12 Ajax Model: Partial UI updates and asynchronous communications

รูปการทำงานแบบ Asynchronous และการ update หน้าเว็บแบบบางส่วน ที่ทำให้การทำงานของผู้ใช้มีประสิทธิภาพมากขึ้น

เบราว์เซอร์ที่สนับสนุนอย่างที่กำลังมาแล้วว่า Ajax เป็นเทคนิคที่อยู่บนพื้นฐานของเทคโนโลยีที่มีอยู่ในปัจจุบัน ไม่มีอะไรใหม่ จึงทำให้โปรแกรมเบราว์เซอร์ที่เป็นที่นิยมกันอยู่ในปัจจุบันสามารถทำงานร่วมกับ Ajax ได้

- Apple Safari 1.2 หรือใหม่กว่า
- Microsoft Internet Explorer 4.0 หรือใหม่กว่า
- Mozilla Firefox 1.0 หรือใหม่กว่า
- Netscape 7.1 หรือใหม่กว่า
- Opera 7.6
- Konqueror

ข้อดีของ Ajax

1. ตอบสนองต่อผู้ใช้ได้อย่างรวดเร็วเนื่องจากการ Update แบบบางส่วน
2. การแสดงผลที่เร็วกว่า ไม่ต้อง Refresh หน้าจอใหม่ทุกครั้งอีกทั้งข้อมูลที่ส่งไป-กลับไม่ได้ส่งไปทั้งหน้า ทำให้กิน bandwidth น้อยกว่า
3. ผู้ใช้ไม่ต้องหยุดรอคอยการประมวลผลของ Server เนื่องจากการติดต่อแบบ Asynchronous รองรับกับเบราว์เซอร์หลักๆที่สามารถใช้ JavaScript ได้
4. ทำให้การประมวลผลที่ Server มีความรวดเร็วขึ้นเนื่องจากการประมวลผลที่ Server ลดลงไม่ต้องทำการติดตั้ง หรือใช้ Plugs-in
5. ไม่ยึดติดกับ Platform หรือภาษาที่ใช้ในการเขียนโปรแกรม
6. เป็นเทคโนโลยีใหม่ที่ไม่ได้เป็นของนักพัฒนาเว็บแอปพลิเคชันคนใด นั่นคือทุกคนมีสิทธิ์เข้ามาพัฒนาแอปพลิเคชันตัวนี้

สรุปก็คือ Ajax ไม่ใช่เทคโนโลยีล่าสุดของการพัฒนาเว็บแอปพลิเคชัน แต่มันประกอบไปด้วยเทคโนโลยีที่เราคุ้นเคยกันคืออยู่แล้ว แต่ถูกจับมารวบรวมอยู่ด้วยกัน Ajax ในวันนี้มีแนวโน้มที่จะเติบโตอย่างรวดเร็วในอนาคตอันใกล้ เพราะแม้แต่ Google ที่ได้รับการยอมรับว่ามีการพัฒนาเรื่องของเทคโนโลยีทางเว็บก้าวหน้าอย่างมากยังนำ Ajax มาใช้ในแอปพลิเคชันของตน เราในฐานะของนักพัฒนาชาวไทย เราคงต้องหันมาสนใจและนำ Ajax มาใช้มากขึ้น